

AI & LLM FUNDAMENTALS • SESSION 7

Application Development & Tool Integration

From prompt to product — the four pieces that turn an LLM into an application.

THE PROBLEM

A bare LLM, dropped into your support chat

Customer:

"Where's my order #A7731, and can I return part of it?"

Bare LLM:

"Shipped Tuesday, arrives Thursday — and yes, you have 30 days to return it!"

Plausible. Confident. Entirely invented.

It never looked up the order, never read your policy, and handed your UI a paragraph it can't act on.

One moment — four failures.

- ✗ Reply your code can't parse
- ✗ Invented the order status
- ✗ Invented the return policy
- ✗ One-shot guess, no recovery

One disease, four cures

1

Structured output

Reply your code can't parse → a typed object it can

2

Tool / function calling

Invented status → call your real systems

3

Retrieval (RAG)

Invented policy → ground it in your data

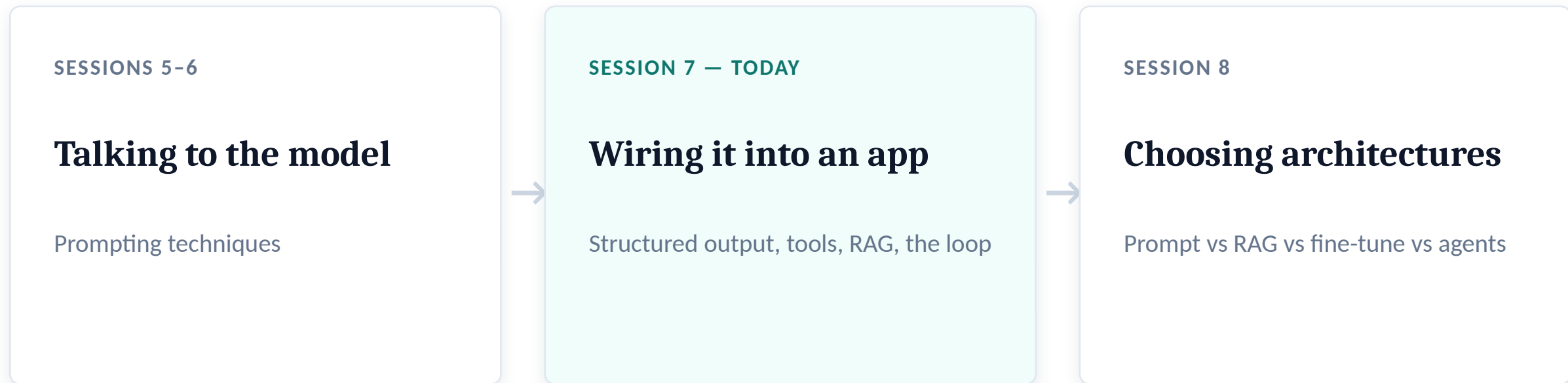
4

Orchestration — the loop

One-shot guess → sequence, recover, stop

SERIES CONTEXT

Where this session sits



S7 teaches how to build the pieces. S8 teaches which to reach for — so RAG and agents appear in both, at different altitudes.

Software talks through contracts

The contract problem

A REST endpoint, a function signature, a DB schema — every block expects a fixed shape.

You can't feed free prose into the next block and hope it copes.

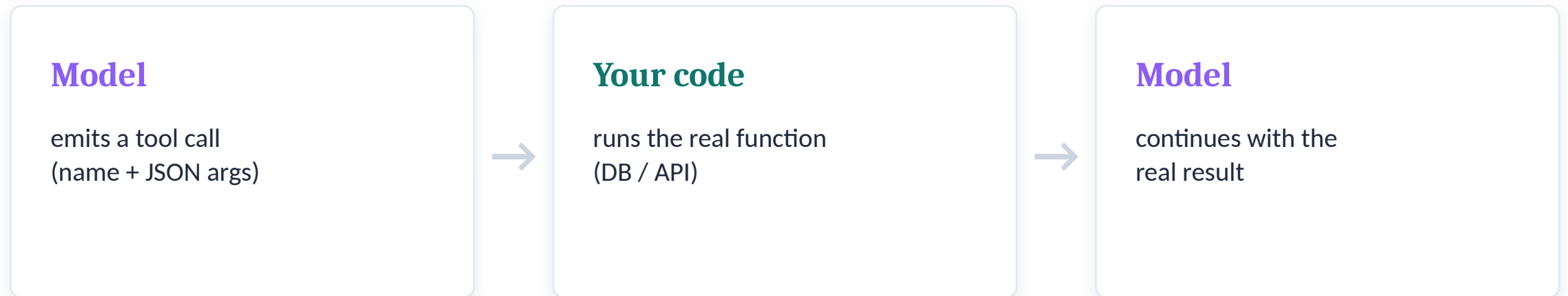
So the model must **fill out a form, not write a letter.**

```
# prose your code can't use:
"Sure! A7731 is on its way..."

# a contract it CAN:
SupportReply(
    intent='order_status',
    order_id='A7731',
    needs_clarification=True,
    message='...'
)

# Session 3's constrained decoding,
# now serving an app contract.
```

Let the model reach your systems



The model is the decision layer; your code is the execution layer. It never touches your DB — that's the whole security story in one line.

ReAct → function calling

ReAct (the research idea)

```
Thought: I need the status  
Action: get_order  
Action Input: A7731  
Observation: delivered  
Thought: now I can answer
```

```
// 'Action' parsed out of  
// free-form text
```

Function calling (production)

The API guarantees a well-formed call — no regex out of prose.

A tool call is structured output aimed at your code.

Same lineage as the ART technique in your reading.

Two knowledge gaps, one fix

Temporal

The model's knowledge is frozen — it doesn't know the present world.

Private

It has never seen your data — policies, orders, docs.

The fix: retrieve relevant text at query time and put it in context. Grounded, current, cited.

Open-book exam — not smarter, just allowed to check its notes.

A question and its answer don't look alike

QUESTION

"Can I return an opened item?"

THE DOCUMENT THAT ANSWERS IT

"Unsealed merchandise may be exchanged within 14 days."

Zero shared words. A human bridges it instantly; keyword search fails. Even embeddings struggle — questions and answers live in different regions of vector space.

Easy for a human in half a second. A research field for a machine.

Each fix patches one gap

1

Embeddings match meaning, not words — 'budget phone' ≈ 'affordable smartphone'

2

Asymmetric models / HyDE train on Q→passage, or embed a hypothetical answer instead of the question

3

Hybrid search (+ BM25) recover exact strings — SKUs, codes — that embeddings lose

4

Reranking a cross-encoder re-scores the shortlist for precision

Pipeline: hybrid → RRF fusion → cross-encoder rerank → top-k → LLM
Session 8)

(RAG vs fine-tune vs long-context →

LAYER 4 • ORCHESTRATION

**An agent is a while-loop around an LLM
with tools, retrieval, and a stopping condition.**

call model → it requests a tool → execute → feed result back → repeat → stop

How does it decode the execution sequence? At runtime, from the query — the way you plan a workflow, but on the fly.

Decide-as-you-go vs plan-then-do

ReAct

Decide as you go

- Thought → Action → Observation, looped
- One LLM call per step
- Adaptive — pivots when a tool surprises it
- Like human trial-and-error
- Risk: can loop or drift

Plan-and-Execute

Plan upfront, then do

- Planner writes the steps; Executor runs them
- Predictable, inspectable, cheaper
- Like project management
- Rigid — needs explicit replanning on failure

The loop holds the resilience

Validate & retry

Output fails its schema or a tool errors
→ feed the error back, try again.

Ambiguity handling

“Which items?” — ask a clarifying question vs assume. A design decision.

Stop conditions

Max steps, no-progress detection, cost ceilings — or it's an expensive random walk.



DEMO

The support assistant, end to end

hallucination → structured output → tools → RAG → the loop → retry — all in one run

One line of autolog → a timeline

```
import mlflow.pydantic_ai
mlflow.pydantic_ai.autolog()

# every run → a trace
```

The span tree

```
Agent.run
├─ LLM request
│   └─ tool: get_order
├─ LLM request
│   └─ tool: get_policy
└─ LLM request → final
```

Why it matters

Every LLM call, tool call, latency, and token count — nested, on a timeline.

The loop is no longer a black box; you can see it think.

Callback to Session 4 — this is your monitoring story, now for agents.

We hand-wired two tools. Now scale it.

The N×M problem

50 tools × 3 apps × 3 models = bespoke glue everywhere. Every model needs custom wiring for every tool.

MCP: N + M

Each tool and each model implements one protocol — and they all interoperate. Glue collapses.

A standard, not an API

Think of it as...

USB-C for AI

one port, any device.

LSP for IDEs

one protocol, any editor × any language server.
(MCP was inspired by it.)

Don't confuse it with function calling

Function calling: the model can call one tool you wired by hand.

MCP: a standard so any model talks to any tool — discoverable, swappable. Built on JSON-RPC 2.0; adopted across the industry.

Handshake → discovery → call

1

Handshake

initialize — agree the protocol version & capabilities before any data flows

Like a TLS / WebSocket negotiation

2

Discovery

tools/list — the server hands back a catalog: each tool with a JSON Schema

“The model reads the menu” (like OpenAPI)

3

Call

tools/call — schema-validated arguments, gated by your approval

Same decision/execution split as Layer 2

Handshake and discovery are TWO separate steps — the one thing a sharp attendee will catch.



DEMO

Same agent, via MCP — then Copilot

The trace shows `list_tools` (discovery) and `call_tool` (call). Then: your Copilot is an MCP host too — it performs the same handshake and queries the tool list.

RECAP

Same failure, four fixes

Unparseable paragraph



Structured output

Invented status



Tool calling

Invented policy



Retrieval (RAG)

One-shot guess



The loop

NEXT • SESSION 8

**You have the building blocks.
Next: which to reach for, and when.**

Prompting vs RAG vs fine-tuning vs agents — the architecture decisions.



Questions?

github.com/girishponkiya/AI-LLM-Fundamentals-Workshop-Series